

43

AUTOMATED TESTS

5

DEFECTS FOUND & VERIFIED

4

TEST CATEGORIES

Overview

SwitchFW QA is a self-contained Python project that simulates an Ethernet switch's management-plane firmware and puts it through the full QA workflow: reviewing requirements, designing test cases, building an automation framework, finding real-looking defects, and verifying that fixes actually close them.

No physical hardware required — built to be cloned and run end-to-end in minutes.

Architecture

1

Mock Firmware — `dut/firmware.py`

Simulated switch state (ports, VLANs, counters, thermal sensor) standing in for real device firmware, deliberately seeded with 5 realistic defects.



2

TCP / JSON Transport — `dut/server.py` + `client.py`

A management interface mirroring real device CLIs (Telnet/console-style), plus a Pythonic automation client — the layer test code is written against.



3

QA Automation — `tests/` + `bugtracker/`

43 pytest cases across functional/regression/performance/security, automatic bug-report generation on failure, and a CLI that verifies fixes before/after.

Tech Stack

Python 3

pytest

TCP Sockets

JSON

YAML

JUnit XML

pytest-html

Git

Multi-threaded server

Multi-device topologies

Skills Demonstrated

- ▶ **Test design & methodology** — requirement-to-test-case traceability matrix
- ▶ **Test automation framework development** — extensible client library + fixtures
- ▶ **Functional, regression, performance & security testing**
- ▶ **Bug lifecycle management** — auto-reporting, repro steps, fix verification
- ▶ **Networking fundamentals** — VLANs (802.1Q), speed/duplex, counters, link state
- ▶ **Test progress reporting** — JUnit XML, HTML, Markdown summaries

5 Defects Found, Documented & Verified

Each bug is gated behind a toggle, so the same codebase represents both the "buggy" and the "fixed" firmware build — used to demonstrate a full find → report → fix → verify lifecycle.

ID	CATEGORY	DEFECT	WHY IT MATTERS
BUG-001	Regression	Back-to-back speed changes leave operational state stale	Classic firmware race condition: a second config write before the first is acknowledged can desync reported vs. actual hardware state.
BUG-002	Performance	Traffic counter saturates instead of wrapping under load	Hardware counters should wrap cleanly on overflow; saturation silently hides real traffic volume from monitoring tools.
BUG-003	Security	VLAN IDs outside the 802.1Q range (1–4094) accepted	Missing boundary validation can cause undefined switching behavior or interop issues with other vendors' gear.
BUG-004	Regression	Thermal warning never raised despite crossing threshold	Monitoring systems would stay blind to overheating hardware — a real safety/reliability gap.
BUG-005	Security	CLI parser crashes on malformed/missing arguments	A management interface crashable by bad input is a robustness defect.

Test Suite Breakdown

CATEGORY	TESTS	FOCUS
Functional	23	Baseline correctness of core features in isolation (port status, speed, VLAN, version, topology, end-to-end wire protocol)
Regression	4	Re-runs exact historical repros so previously found defects can't silently resurface
Performance	4	Behavior under sustained load and timing budgets across repeated operations
Security	12	Boundary values, malformed/fuzzed input, and verification that defensive controls hold

Repository Layout

```
switchfw-qa/  
├─ dut/           mock firmware, TCP server, automation client, topology loader  
├─ tests/         functional/ · regression/ · performance/ · security/  
├─ bugtracker/    bug auto-reporter, fix-verification CLI, summary generator  
├─ bugs/          auto-generated bug reports (populated by running pytest)  
├─ reports/       JUnit XML, HTML report, Markdown summary (auto-generated)  
├─ docs/          traceability matrix, networking primer  
└─ topology.yaml example 2-switch test topology
```

Full source, setup instructions, and live results in this repository's [README.md](#).

Quickstart: `pip install -r requirements.txt && pytest -v`